

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 06/Part 02: Rule of 3

Course taught by: Zubair Irshad
Courtesy of some slides: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Introduction
2. Destructor
3. Copy Constructor
4. Copy Assignment Operator
5. Summary



Introduction

- In C++, the Rule of 3 is a convention (best practice). It states that:
- If a class needs to define one of the following:
 1. Destructor,
 2. Copy Constructor
 3. Copy Assignment Operator

then it needs to explicitly define all 3 in order to have exception safe coding.

The rule of 3 is a convention or a best practice in C++.



Rule of 3 Motivation

- **Destructor** – to free the resource.
- **Copy Constructor** – to make a deep copy.
- **Copy Assignment Operator** – to handle assignment correctly.

We need Rule of 3 to avoid problems like:

- Shallow Copy
- Double deletion
- Two objects sharing the same memory
- Memory leaks



Rule of 3 - Destructors

As discussed in previous sessions, a **destructor** is a function that is

- called when an object goes out of scope or,
- is explicitly deleted.

It is responsible for cleaning up resources that the object may have acquired during its lifetime, such as dynamically allocated memory.



Example of Destructor

```
class Rectangle {  
private:  
    int *width, *height;  
  
public:  
    // Constructor  
    Rectangle(int w, int h) {  
        width = new int(w);  
        height = new int(h);  
        cout << "Rectangle created: "  
        << *width << "x" << *height << endl;  
    }  
  
    // Destructor  
    ~Rectangle() {  
        delete width;  
        delete height;  
        cout << "Rectangle destroyed" << endl;  
    }  
};
```



Rule of 3 – Copy Constructors

- A constructor that creates a new object as a **copy of an existing object**.
- It is called when an object is passed by value, returned from a function, or explicitly copied.
- If the class manages resources (like **dynamic memory**), a **default copy constructor** may lead to **shallow copies**, causing resource management issues.
- We learnt how to make **custom copy constructors**!



Example of custom copy constructor

```
class Rectangle {
public:
    int* width;    // pointer to store the address of width
    int* height;   // pointer to store the address of height

    // Constructor (single line allocation + initialization)
    Rectangle(int w, int h) {
        width = new int(w);    // allocate and initialize
        height = new int(h);   // allocate and initialize
    }

    // Deep copy constructor (step-by-step for clarity)
    Rectangle(const Rectangle& other) {
        width = new int;        // allocate new memory and point to that new address
        *width = *other.width;  // copy the value
        height = new int;       // allocate new memory and point to that new address
        *height = *other.height; // copy the value
    }

    // Destructor
    ~Rectangle() {
        delete width;
        delete height;
    }
};
```




Recap Copy Constructor (Two ways!)

- This causes the default copy constructor for the Distance class to perform a member-by-member copy of rect1 into rect2.

`Rectangle rect2 = rect1;`

- A different format with exactly the same effect: rect1 is copied member-by-member into rect3

`Rectangle rect3(rect1);`



Recap Copy Constructor (Two ways!)

- We didn't handle the case where you want to copy already initialized class into another initialized class, let's look at this example:

Rectangle rect1(10, 20);

Rectangle rect2(5, 15);

rect2 = rect1;

// we want to write a Copy Assignment Operator

3rd Rule



Rule of 3 – Copy Assignment Operator

Copy Assignment Operator:

- **Definition:** Special function that overloads the = operator.
- **Why needed:** Assigns the values of one object to another **after both already exist**.
- Like the copy constructor, if the class manages resources, the **default assignment operator** may lead to shallow copies.
- **Best Practice:** If your class manages resources, always define it explicitly (along with destructor and copy constructor).



Copy Constructor vs Copy Assignment Operator

- **Copy Constructor:** A special constructor that initializes a new object as a copy of an existing object.

```
Rectangle(const Rectangle& other);
```

- Called when **a new object** is created from an existing object.
- **Copy Assignment Operator:** A special assignment operator that assigns the contents of one existing object to another existing object (both objects are already created).

```
Rectangle& operator=(const Rectangle& other);
```

- Called when **an existing object** is assigned to another existing object.



Copy Constructor vs Copy Assignment Operator

```
// Copy Constructor
Rectangle(const Rectangle& other) {
    cout << "Copy Constructor called\n";
    width  = new int(*other.width);
    height = new int(*other.height);
}
```

```
// Copy Assignment Operator
void operator=(const Rectangle& other) {
    cout << "Copy Assignment called\n";

    // Free the old memory
    delete width;
    delete height;

    // Allocate new memory and copy values
    width  = new int(*other.width);
    height = new int(*other.height);
}
```

Key takeaway: Note that the assignment operator needs to deallocate memory since it is called on an already initialized object



Example: Rule of 3 not followed

```
#include <iostream>
using namespace std;

class Rectangle {
    int* width;
    int* height;
public:
    // Constructor
    Rectangle(int w=0, int h=0) {
        width = new int(w);
        height = new int(h);
    }

    // Destructor (but no copy constructor or assignment operator)
    ~Rectangle() {
        delete width;
        delete height;
    }

    void print() const {
        cout << "Rectangle: " << *width << " x " << *height << endl;
    }
};
```

```
int main() {
    cout << "Creating r1...\n";
    Rectangle r1(3, 4);
    r1.print();

    cout << "\nCopy constructing r2 from r1...\n";
    Rectangle r2 = r1;    // uses default copy constructor (shallow copy!)
    r2.print();

    cout << "\nAssigning r3 from r1...\n";
    Rectangle r3(10, 20);
    r3 = r1;              // uses default assignment operator (shallow copy!)
    r3.print();

    cout << "\nExiting main, destructors will run...\n";
}
```

Example: Rule of 3 followed!

```
public:
    // Constructor
    Rectangle(int w=0, int h=0) {
        width  = new int(w);
        height = new int(h);
    }

    // Destructor
    ~Rectangle() {
        delete width;
        delete height;
    }

    // Copy Constructor
    Rectangle(const Rectangle& other) {
        cout << "Copy Constructor called\n";
        width  = new int(*other.width);    // deep copy
        height = new int(*other.height);
    }

    // Copy Assignment Operator
    void operator=(const Rectangle& other) {
        cout << "Copy Assignment called\n";
        delete width;                      // free old memory
        delete height;
        width  = new int(*other.width);    // deep copy
        height = new int(*other.height);
    }
}
```

Destructor, Copy Constructor and Copy Assignment Operator all defined

- Both **copy constructor** and **copy assignment** now create independent memory. Important point!
- We won't see problems like:
 - Shallow copies,
 - Double deletes
 - Memory leaks.



Rule of 3 – Problems Reminder

Some of the problems resulting for not following the rule of 3 are:

- **Double Deletion** – deleting an object that has already been deleted due to default copy constructor/copy assignment operator behavior i.e. copying addresses and not creating new addresses
- **Shallow Copy** – creating shallow copies of objects, leading to a modification in one of them, modifying all of them.
- **Memory Leaks** – Improper deletion and release of resources may cause memory leaks.



Rule of 3 - Summary

- The rule of 3 states that: If a class needs to define one of the following:
 - Destructor,
 - Copy Constructor
 - Copy Assignment Operator

then it needs to explicitly define all 3 in order to have exception safe coding.

- Without following the rule of 3, problems such as
 - Double deletion
 - Shallow copy
 - Memory leaks

may happen.